Harmonic analysis in image compression

By: Josh Tabb

Preliminaries - Representing Image Data

Preliminaries - Representing Image Data



Preliminaries - Representing Image Data



$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{i1} & a_{i2} & a_{i3} & \dots & a_{in} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \dots & a_{mn} \end{bmatrix}$$

Preliminaries - Representing Image Data

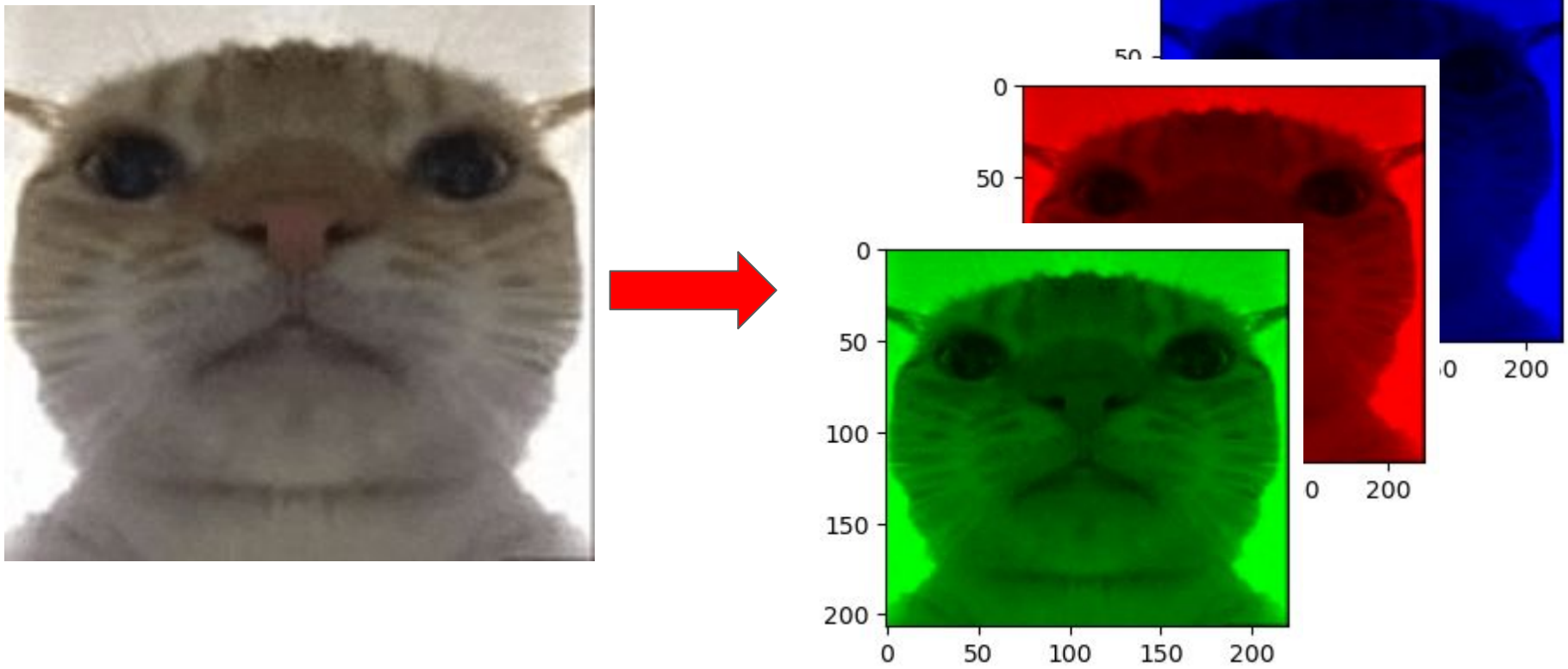


512x512 pixels

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{i1} & a_{i2} & a_{i3} & \dots & a_{in} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \dots & a_{mn} \end{bmatrix}$$

m=n=512

Preliminaries - Representing Image Data



Preliminaries - Representing Image Data



512x512 pixels

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{i1} & a_{i2} & a_{i3} & \cdots & a_{in} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{bmatrix}$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{i1} & a_{i2} & a_{i3} & \cdots & a_{in} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{bmatrix}$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{i1} & a_{i2} & a_{i3} & \cdots & a_{in} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{bmatrix}$$

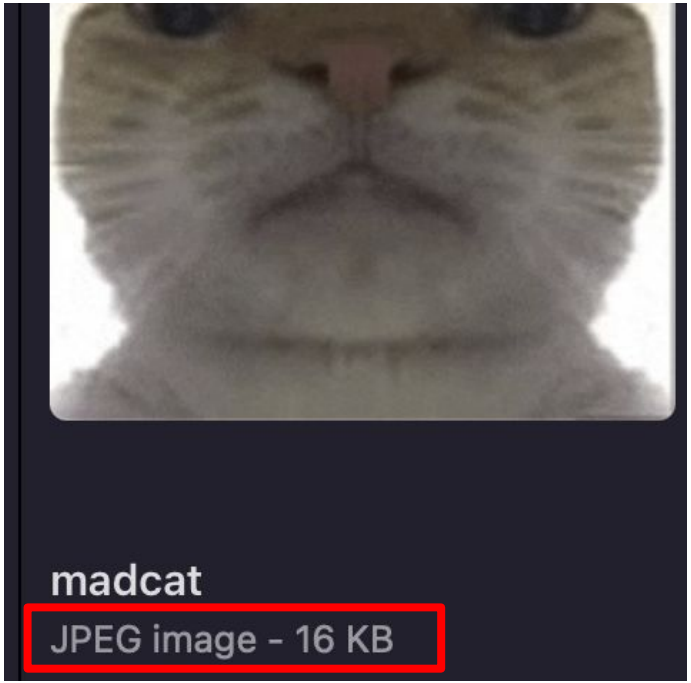
Critical Observation 1

- 512 * 512 * 3 entries of numbers in [0, 255]
- ~ 780,000 bytes of data (~ .78 mB)

Critical Observation 1

- 512 * 512 * 3 entries of numbers in [0, 255]
- ~ 780,000 bytes of data (~ .78 mB)

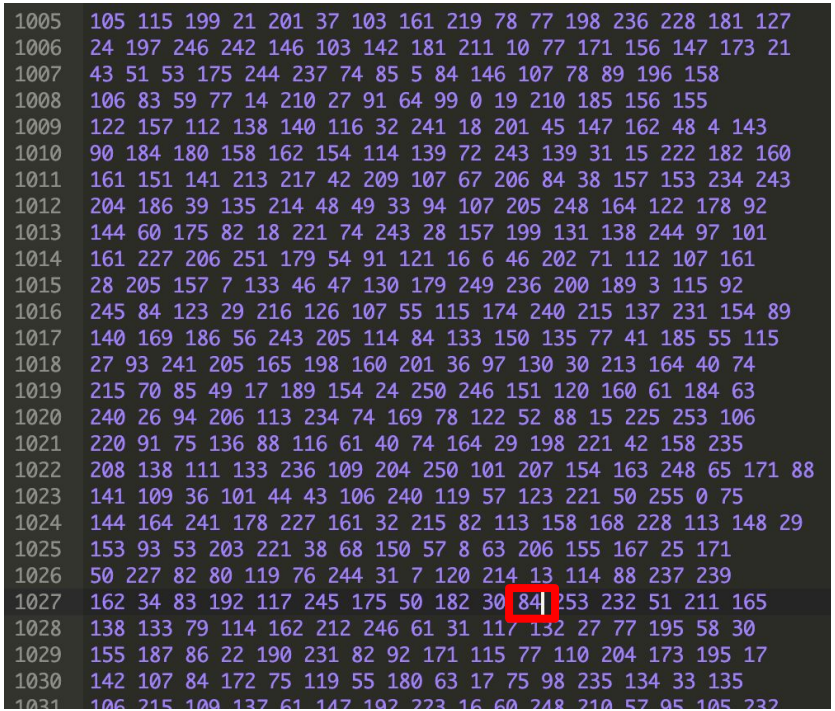
Somehow only using **(1/50)'th** of the "necessary" data!

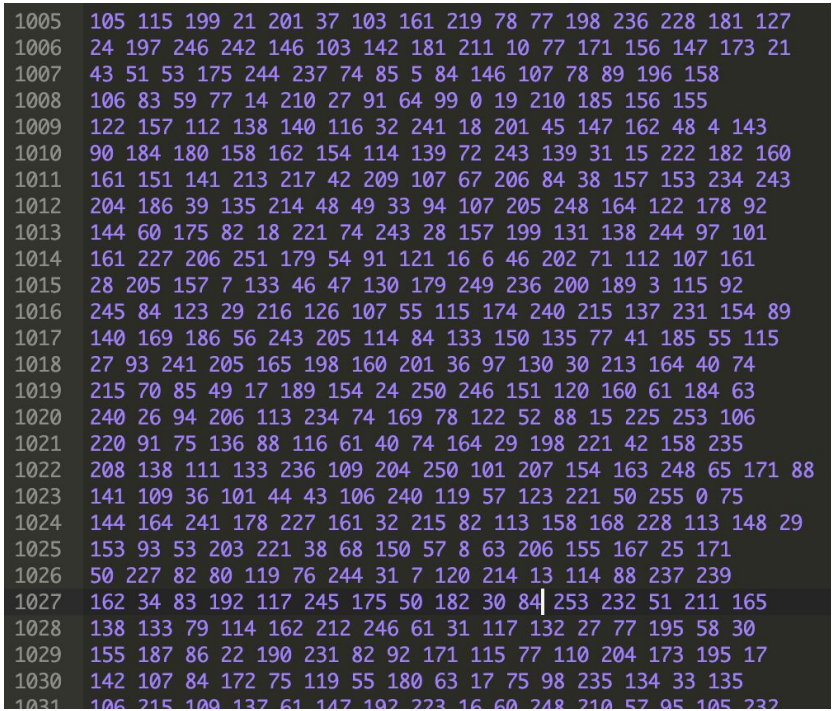


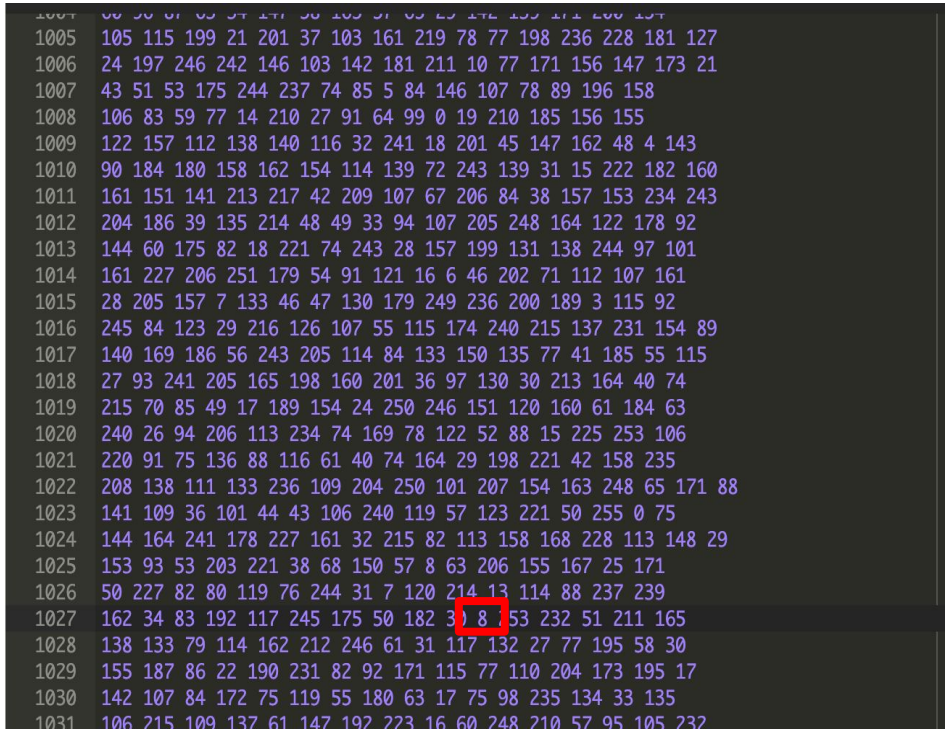
Critical Observation 2

```
255 218 0 12 3 1 0 2 17 3 17 0 63 0 249 148
254 254 100 140 62 216 97 52 208 131 190 104 24 103 32 250
208 33 51 84 12 9 164 3 41 128 229 60 208 2 154 0
107 80 38 200 95 245 170 1 153 166 49 172 120 161 8 76
241 77 130 26 78 71 248 80 75 35 99 84 50 187 158 212
34 25 92 183 94 57 173 17 44 96 63 133 81 40 27 214
144 198 49 31 90 6 153 11 114 105 12 117 179 109 153 104
98 61 23 194 151 98 61 156 210 41 234 123 95 131 181 61
225 59 86 177 48 108 245 205 14 231 204 137 115 90 163 38
116 80 174 224 13 89 29 75 107 23 203 76 4 146 46 58
82 2 133 204 124 30 40 3 18 250 62 181 44 70 29 202
114 120 169 46 229 70 143 39 252 104 37 187 130 198 104 17
110 24 137 233 76 108 188 176 100 123 85 48 26 240 117 226
164 104 207 185 131 57 56 160 104 231 181 75 110 13 0 209
199 106 208 240 220 80 51 138 213 163 198 234 0 228 53 21
229 170 128 231 175 87 154 123 146 97 93 245 63 206 173 16
204 217 133 81 5 9 135 90 162 89 74 65 207 74 98 66
168 224 87 60 206 234 68 192 29 181 193 45 207 78 43 66
165 200 60 213 196 137 148 36 25 53 212 142 9 11 0 203
125 107 67 156 214 181 29 42 36 84 81 173 2 252 149 204
206 148 180 30 203 245 164 85 136 177 64 135 45 12 99 240
113 214 149 199 97 173 199 227 76 77 12 69 203 123 10 119
18 52 173 23 24 61 235 158 76 232 137 183 104 57 21 206
205 145 179 105 242 227 214 145 47 67 98 217 134 57 164 65
126 23 226 129 22 16 156 131 210 128 36 3 34 129 6 78
122 80 49 84 224 30 122 210 11 14 67 215 214 129 15 39
241 166 49 81 136 53 86 2 81 32 199 25 169 176 134 23
```




1005 105 115 199 21 201 37 103 161 219 78 77 198 236 228 181 127
1006 24 197 246 242 146 103 142 181 211 10 77 171 156 147 173 21
1007 43 51 53 175 244 237 74 85 5 84 146 107 78 89 196 158
1008 106 83 59 77 14 210 27 91 64 99 0 19 210 185 156 155
1009 122 157 112 138 140 116 32 241 18 201 45 147 162 48 4 143
1010 90 184 180 158 162 154 114 139 72 243 139 31 15 222 182 160
1011 161 151 141 213 217 42 209 107 67 206 84 38 157 153 234 243
1012 204 186 39 135 214 48 49 33 94 107 205 248 164 122 178 92
1013 144 60 175 82 18 221 74 243 28 157 199 131 138 244 97 101
1014 161 227 206 251 179 54 91 121 16 6 46 202 71 112 107 161
1015 28 205 157 7 133 46 47 130 179 249 236 200 189 3 115 92
1016 245 84 123 29 216 126 107 55 115 174 240 215 137 231 154 89
1017 140 169 186 56 243 205 114 84 133 150 135 77 41 185 55 115
1018 27 93 241 205 165 198 160 201 36 97 130 30 213 164 40 74
1019 215 70 85 49 17 189 154 24 250 246 151 120 160 61 184 63
1020 240 26 94 206 113 234 74 169 78 122 52 88 15 225 253 106
1021 220 91 75 136 88 116 61 40 74 164 29 198 221 42 158 235
1022 208 138 111 133 236 109 204 250 101 207 154 163 248 65 171 88
1023 141 109 36 101 44 43 106 240 119 57 123 221 50 255 0 75
1024 144 164 241 178 227 161 32 215 82 113 158 168 228 113 148 29
1025 153 93 53 203 221 38 68 150 57 8 63 206 155 167 25 171
1026 50 227 82 80 119 76 244 31 7 120 214 13 114 88 237 239
1027 162 34 83 192 117 245 175 50 182 30 84 253 232 51 211 165
1028 138 133 79 114 162 212 246 61 31 117 132 27 77 195 58 30
1029 155 187 86 22 190 231 82 92 171 115 77 110 204 173 195 17
1030 142 107 84 172 75 119 55 180 63 17 75 98 235 134 33 135


1005 105 115 199 21 201 37 103 161 219 78 77 198 236 228 181 127
1006 24 197 246 242 146 103 142 181 211 10 77 171 156 147 173 21
1007 43 51 53 175 244 237 74 85 5 84 146 107 78 89 196 158
1008 106 83 59 77 14 210 27 91 64 99 0 19 210 185 156 155
1009 122 157 112 138 140 116 32 241 18 201 45 147 162 48 4 143
1010 90 184 180 158 162 154 114 139 72 243 139 31 15 222 182 160
1011 161 151 141 213 217 42 209 107 67 206 84 38 157 153 234 243
1012 204 186 39 135 214 48 49 33 94 107 205 248 164 122 178 92
1013 144 60 175 82 18 221 74 243 28 157 199 131 138 244 97 101
1014 161 227 206 251 179 54 91 121 16 6 46 202 71 112 107 161
1015 28 205 157 7 133 46 47 130 179 249 236 200 189 3 115 92
1016 245 84 123 29 216 126 107 55 115 174 240 215 137 231 154 89
1017 140 169 186 56 243 205 114 84 133 150 135 77 41 185 55 115
1018 27 93 241 205 165 198 160 201 36 97 130 30 213 164 40 74
1019 215 70 85 49 17 189 154 24 250 246 151 120 160 61 184 63
1020 240 26 94 206 113 234 74 169 78 122 52 88 15 225 253 106
1021 220 91 75 136 88 116 61 40 74 164 29 198 221 42 158 235
1022 208 138 111 133 236 109 204 250 101 207 154 163 248 65 171 88
1023 141 109 36 101 44 43 106 240 119 57 123 221 50 255 0 75
1024 144 164 241 178 227 161 32 215 82 113 158 168 228 113 148 29
1025 153 93 53 203 221 38 68 150 57 8 63 206 155 167 25 171
1026 50 227 82 80 119 76 244 31 7 120 214 13 114 88 237 239
1027 162 34 83 192 117 245 175 50 182 30 84 253 232 51 211 165
1028 138 133 79 114 162 212 246 61 31 117 132 27 77 195 58 30
1029 155 187 86 22 190 231 82 92 171 115 77 110 204 173 195 17
1030 142 107 84 172 75 119 55 180 63 17 75 98 235 134 33 135


1005 105 115 199 21 201 37 103 161 219 78 77 198 236 228 181 127
1006 24 197 246 242 146 103 142 181 211 10 77 171 156 147 173 21
1007 43 51 53 175 244 237 74 85 5 84 146 107 78 89 196 158
1008 106 83 59 77 14 210 27 91 64 99 0 19 210 185 156 155
1009 122 157 112 138 140 116 32 241 18 201 45 147 162 48 4 143
1010 90 184 180 158 162 154 114 139 72 243 139 31 15 222 182 160
1011 161 151 141 213 217 42 209 107 67 206 84 38 157 153 234 243
1012 204 186 39 135 214 48 49 33 94 107 205 248 164 122 178 92
1013 144 60 175 82 18 221 74 243 28 157 199 131 138 244 97 101
1014 161 227 206 251 179 54 91 121 16 6 46 202 71 112 107 161
1015 28 205 157 7 133 46 47 130 179 249 236 200 189 3 115 92
1016 245 84 123 29 216 126 107 55 115 174 240 215 137 231 154 89
1017 140 169 186 56 243 205 114 84 133 150 135 77 41 185 55 115
1018 27 93 241 205 165 198 160 201 36 97 130 30 213 164 40 74
1019 215 70 85 49 17 189 154 24 250 246 151 120 160 61 184 63
1020 240 26 94 206 113 234 74 169 78 122 52 88 15 225 253 106
1021 220 91 75 136 88 116 61 40 74 164 29 198 221 42 158 235
1022 208 138 111 133 236 109 204 250 101 207 154 163 248 65 171 88
1023 141 109 36 101 44 43 106 240 119 57 123 221 50 255 0 75
1024 144 164 241 178 227 161 32 215 82 113 158 168 228 113 148 29
1025 153 93 53 203 221 38 68 150 57 8 63 206 155 167 25 171
1026 50 227 82 80 119 76 244 31 7 120 214 13 114 88 237 239
1027 162 34 83 192 117 245 175 50 182 30 8 53 232 51 211 165
1028 138 133 79 114 162 212 246 61 31 117 132 27 77 195 58 30
1029 155 187 86 22 190 231 82 92 171 115 77 110 204 173 195 17
1030 142 107 84 172 75 119 55 180 63 17 75 98 235 134 33 135







Guiding Question:

Guiding Question:

What exactly is being stored inside this matrix?

Compression

Compression

- Lossy = Allows some loss of features
  - E.g. JPEG, JP2, HEIC

Compression

- Lossy = Allows some loss of features
  - E.g. JPEG, JP2, HEIC
- Lossless = No loss of features
  - E.g. PNG, GIF

Naive approaches to image compression

Naive approaches to image compression





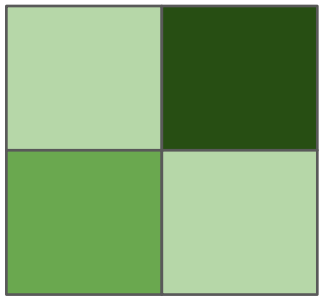
$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{i1} & a_{i2} & a_{i3} & \cdots & a_{in} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{bmatrix}$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{i1} & a_{i2} & a_{i3} & \cdots & a_{in} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{bmatrix}$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{i1} & a_{i2} & a_{i3} & \cdots & a_{in} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{bmatrix}$$

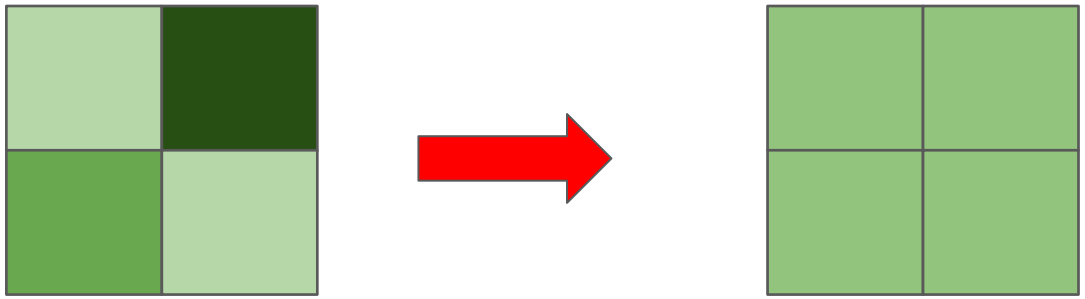
Idea 1: Reduce detail

Idea 1: Reduce detail



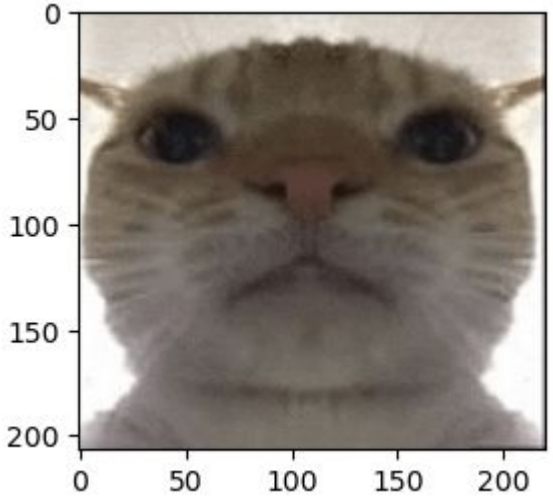
Take group of 4 pixels

Idea 1: Reduce detail

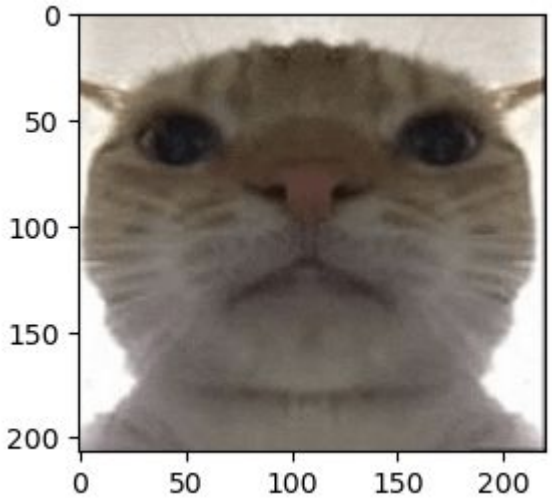


Replace each by
the average color

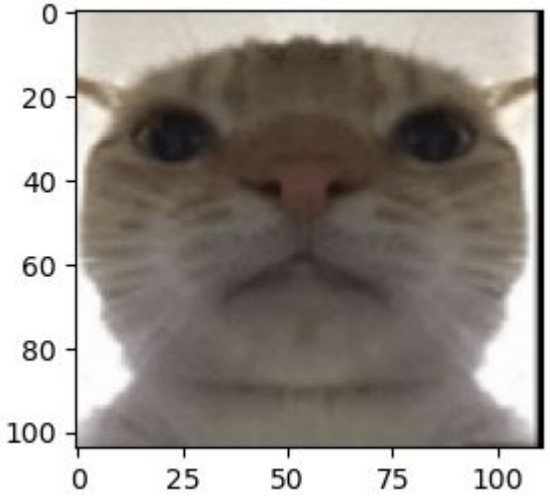
Idea 1: Reduce detail



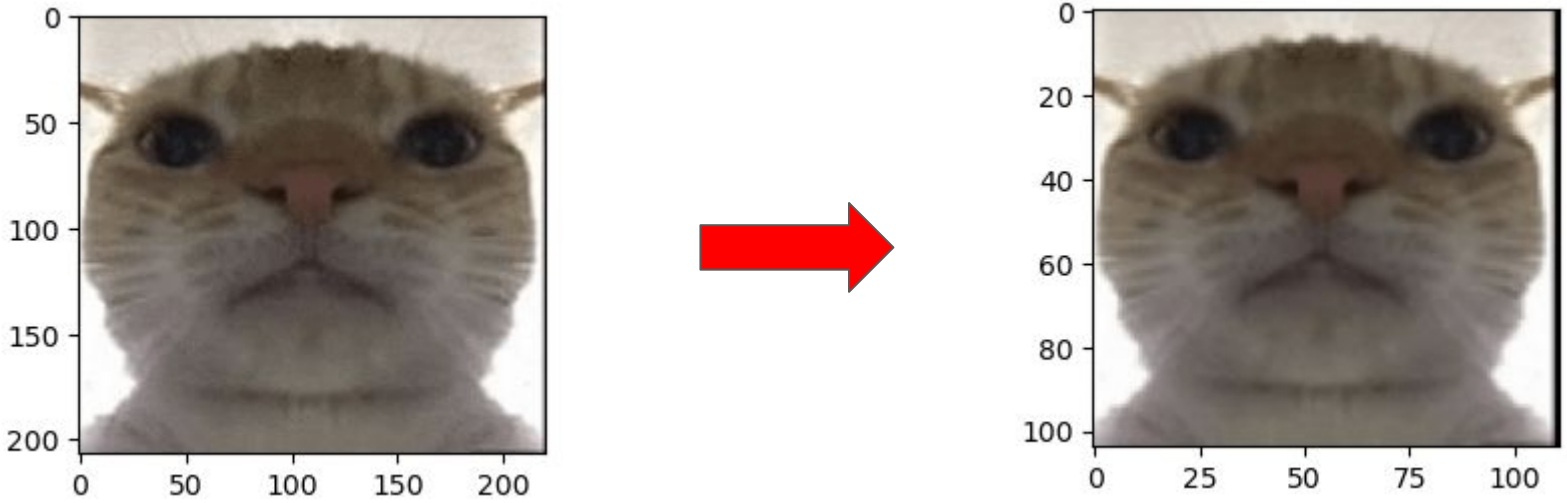
Idea 1: Reduce detail







Idea 1: Reduce detail



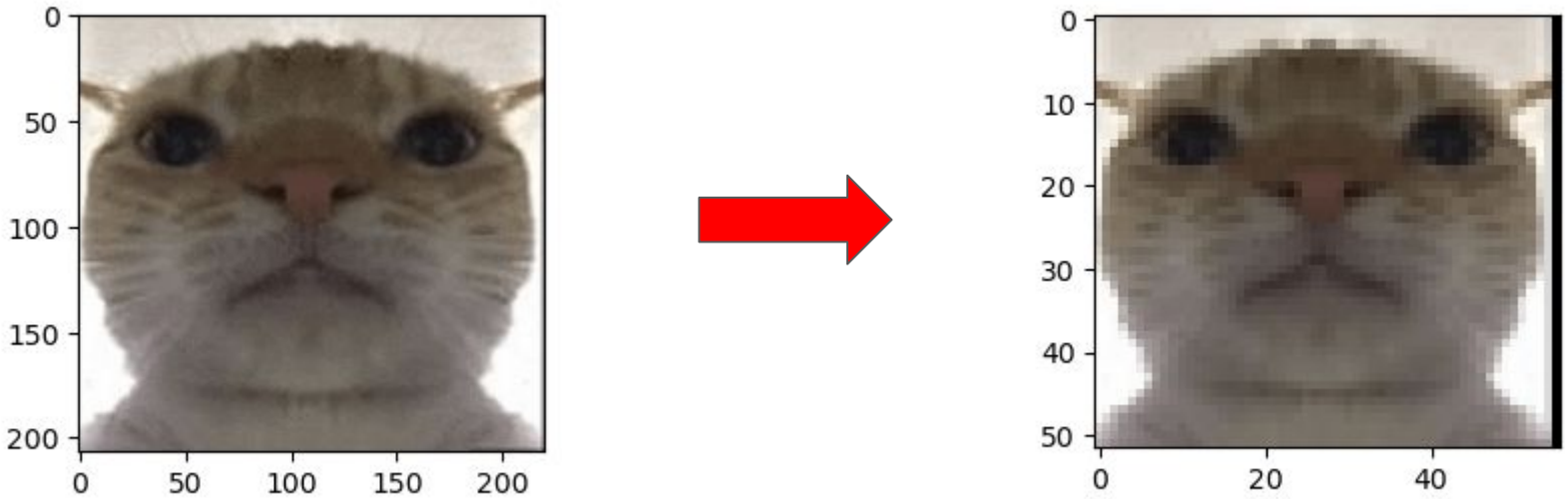
We’ve compressed at a ratio 4:1 with no visible defects!

Problem 1:

- What happens if we try to get a better compression ratio?

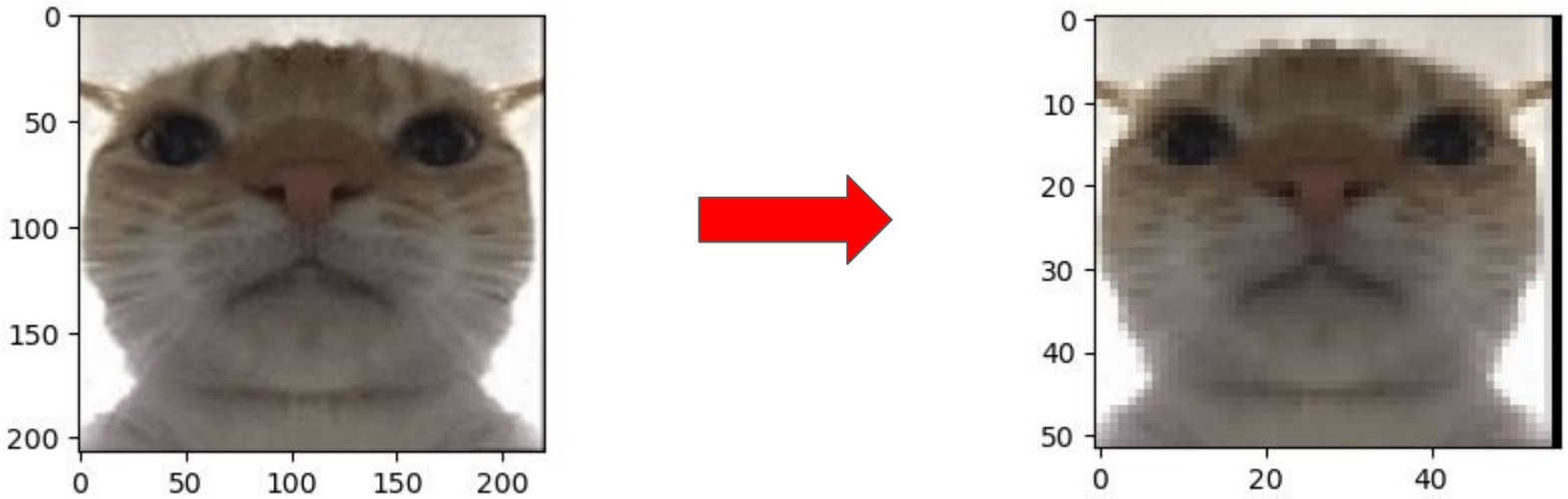
Problem 1:

- What happens if we try to get a better compression ratio?



Problem 1:

- What happens if we try to get a better compression ratio?



Already start getting visual defects when we try to compress by 16:1

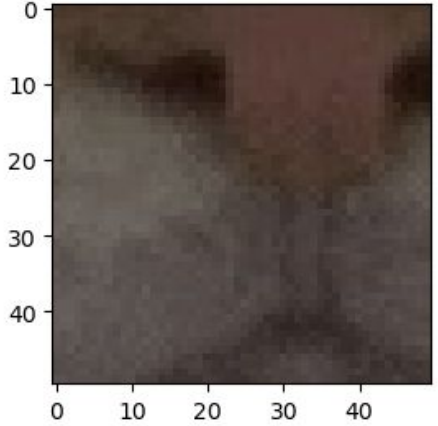
Idea 2: “Smart” detail reduction

Idea 2: “Smart” detail reduction



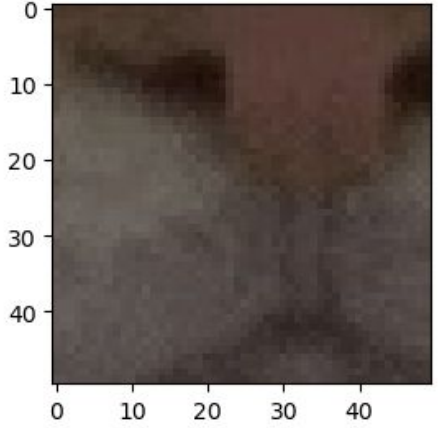
Idea 2: “Smart” detail reduction

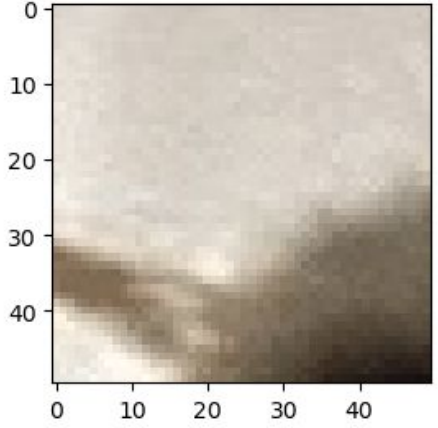




Idea 2: “Smart” detail reduction







Idea 2: “Smart” detail reduction



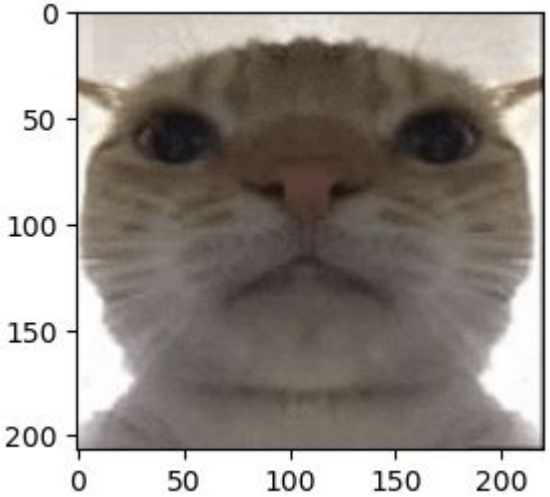
(9:1 reduction faces significant problems)

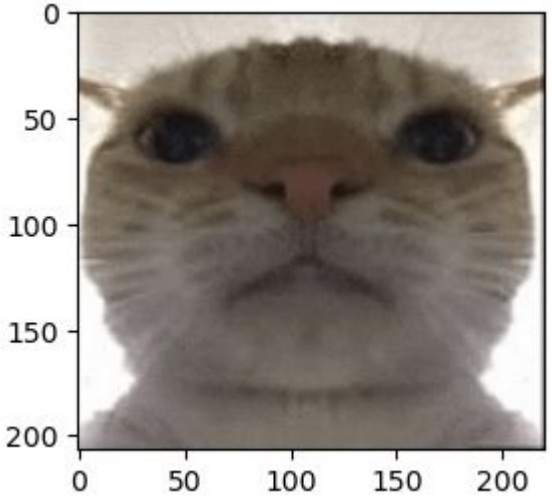
Idea 2: “Smart” detail reduction



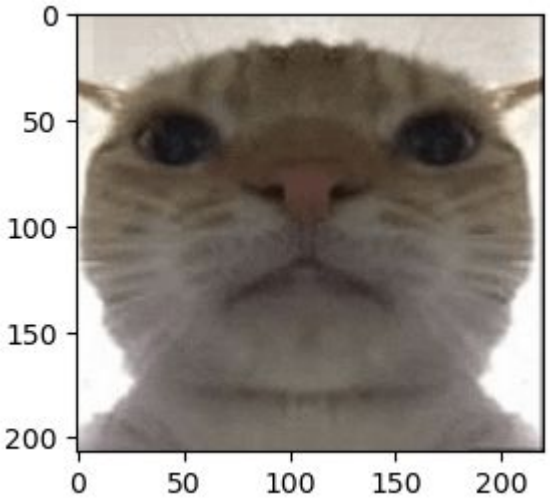
(9:1 reduction sees minimal change)

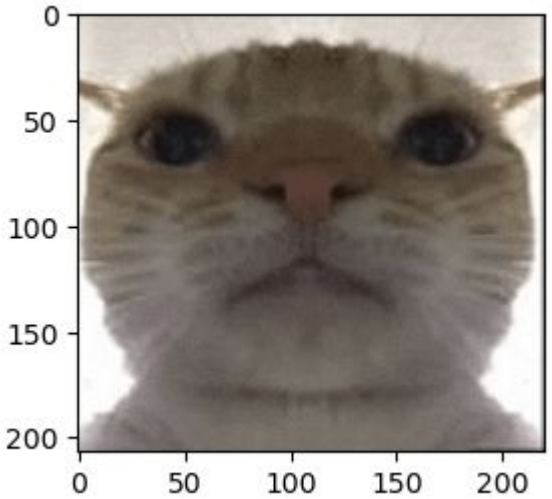
Idea 2: “Smart” detail reduction





Idea 2: “Smart” detail reduction





(The new is on the LEFT - could you tell the difference?)

Problem 2:

- How do we know which boxes to use larger reductions for?

Problem 2:

- How do we know which boxes to use larger reductions for?
  - In other words, what is the threshold, **ε(N)**?

Problem 2:

- How do we know which boxes to use larger reductions for?
  - In other words, what is the threshold, **ε(N)**?
- The gain in ratio is dependent on such boxes existing!

Problem 2:

- How do we know which boxes to use larger reductions for?
  - In other words, what is the threshold, **ε(N)**?
- The gain in ratio is dependent on such boxes existing!

Idea: Find a way to check all dyadic boxes in an efficient way…

Wavelets

Images as functions

Images as functions

- Assume for simplicity that we work with B/W images
- Let $\Omega := [0,1]^2$, let $f: \Omega \rightarrow [0,1)$
- Let D_n:= Collection of dyadic squares of length 2^{-n}

Images as functions

- Assume for simplicity that we work with B/W images
- Let $\Omega := [0,1]^2$, let $f: \Omega \rightarrow [0,1)$
- Let D_n:= Collection of dyadic squares of length 2^{-n}
  - E.g. D_9 has 512*512 squares

Images as functions

If we know…

$$f = \sum_{Q \in D_9} c_Q \chi_Q$$

Images as functions

If we know…

$$f = \sum_{Q \in D_9} c_Q \chi_Q$$

True if and only if f corresponds to the **data of a 512x512 image**!

Averaging algorithm, restated

Averaging algorithm, restated

If **Tf** takes averages over 2x2 squares:

Averaging transform, restated

If **Tf** takes averages over 2x2 squares:

$$Tf = \sum_{Q' \in D_8} c_{Q'} \chi_{Q'}$$

Averaging transform, restated

If **Tf** takes averages over 2x2 squares:

$$Tf = \sum_{Q' \in D_8} c_{Q'} \chi_{Q'}$$

$$c_{Q'} = \frac{1}{4} \sum_{Q \in D_9 : Q \subset Q'} c_Q \qquad c_Q = 4^8 \int_{Q'} f(x)\, dx$$

Averaging transform, restated

- Use E_i to denote the average over cubes in D_i

Averaging transform, restated

- Use E_i to denote the average over cubes in D_i

$$E_i f = \sum_{Q \in D_i} c_Q \chi_Q$$

$$c_Q = \frac{1}{|Q|} \int_Q f(x)\, dx$$

(Some of you might recognize this as the conditional expectation operator over D_i)

Averaging transform, restated

$$
\begin{aligned}
f &= E_9 f \\
&= E_9 f - E_8 f + E_8 f \\
&= (E_9 f - E_8 f) + \cdots + E_0 f
\end{aligned}
$$

Averaging transform, restated

- Representation might seem suboptimal

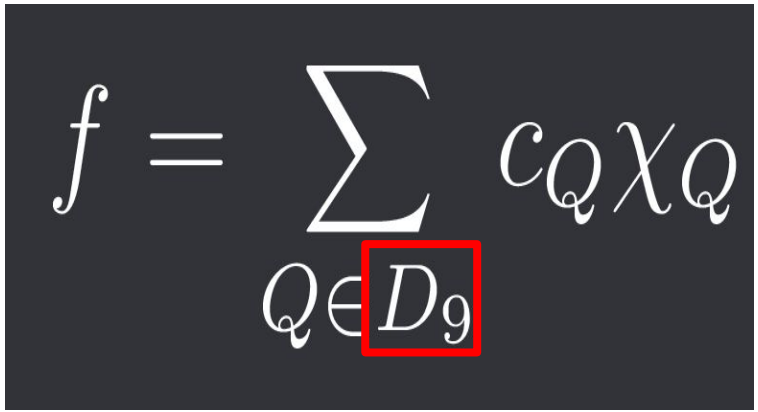
Averaging transform, restated

- Representation might seem suboptimal

$$f = \sum_{Q \in D_9} c_Q \chi_Q$$

Averaging transform, restated

- Representation might seem suboptimal



4^9 coefficients to store

Averaging transform, restated

$$E_9 f - E_8 f = \sum_{Q \in D_9} (c_{Q'} - c_Q) \chi_Q$$

Averaging transform, restated

$$E_9 f - E_8 f = \sum_{Q \in D_9} (c_{Q'} - c_Q) \chi_Q$$



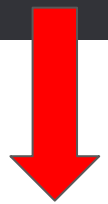
Still have 4^9 coefficients to store!

Averaging transform, restated

$$(E_9 f - E_8 f) + \cdots + E_0 f$$

Averaging transform, restated

$$(E_9 f - E_8 f) + \cdots + E_0 f$$



$(4^9 + \ldots + 4 + 1) \sim (4/3)4^9$
coefficients to store!

Haar wavelet trick

$$E_9 f - E_8 f = \sum_{Q \in D_9} (c_{Q'} - c_Q) \chi_Q$$

Haar wavelet trick

$$E_9 f - E_8 f = \sum_{Q \in D_9} (c_{Q'} - c_Q) \chi_Q$$

$$\chi_{Q'}(E_9 - E_8)(f) = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Haar wavelet trick

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} = \alpha \begin{bmatrix} + & - \\ + & - \end{bmatrix} + \beta \begin{bmatrix} - & - \\ + & + \end{bmatrix} + \gamma \begin{bmatrix} - & + \\ + & - \end{bmatrix},$$

Haar wavelet trick

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} = \alpha \begin{bmatrix} + & - \\ + & - \end{bmatrix} + \beta \begin{bmatrix} - & - \\ + & + \end{bmatrix} + \gamma \begin{bmatrix} - & + \\ + & - \end{bmatrix},$$

Reduce from $(4/3)4^9 \rightarrow 4^9$ coefficients!

Haar wavelet trick

Let Ψ be the collection of functions:

$$\psi^{(1)} = \phi_{[0,1/2)\times[0,1)} - \phi_{[1/2,1)\times[0,1)},$$

$$\psi^{(2)} = \phi_{[0,1)\times[0,1/2)} - \phi_{[0,1)\times[1/2,1)},$$

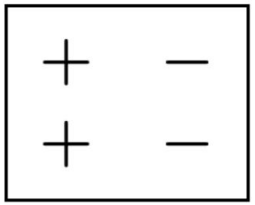
$$\psi^{(3)} = \phi_{[0,1/2)\times[0,1/2)} - \phi_{[0,1/2)\times[1/2,1)} - \phi_{[1/2,1)\times[0,1/2)} + \phi_{[1/2,1)\times[1/2,1)}$$

Haar wavelet trick

Let Ψ be the collection of functions:

$$\psi^{(1)} = \phi_{[0,1/2)\times[0,1)} - \phi_{[1/2,1)\times[0,1)},$$





$$\psi^{(2)} = \phi_{[0,1)\times[0,1/2)} - \phi_{[0,1)\times[1/2,1)},$$

$$\psi^{(3)} = \phi_{[0,1/2)\times[0,1/2)} - \phi_{[0,1/2)\times[1/2,1)} - \phi_{[1/2,1)\times[0,1/2)} + \phi_{[1/2,1)\times[1/2,1)}$$

Haar wavelet trick

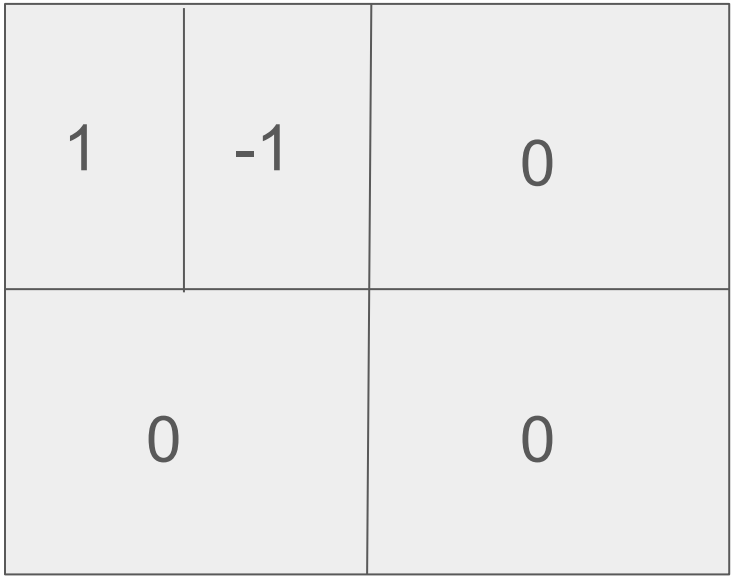
For each function $\psi := \psi^{(i)}$, define

$$\psi_{j,k} = 2^k \psi(2^k x - j)$$

Haar wavelet trick

E.g. if $\psi := \psi^{(1)}$

$\psi_{(0,1)} =$



Haar wavelet trick

$$E_9 f - E_8 f = \sum_{Q \in D_9} d_Q \chi_Q$$

$$= \sum_{j \in \mathbb{Z}^2} \alpha_{j,8} \psi^{(1)}_{j,8} + \beta_{j,8} \psi^{(2)}_{j,8} + \gamma_{j,8} \psi^{(3)}_{j,8}$$

Haar wavelet trick

$$f = \sum_{k \geq 0} \sum_{j \in \mathbf{Z}^2(k)} \sum_{\psi \in \Psi} c_{j,k,\psi} \psi_{j,k} + P_0 f;$$

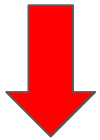
But how does this help us?

But how does this help us?

Want: a lossy compression P’ of image P

But how does this help us?

Want: a lossy compression P’ of image P



In the new setting: find an approximation f’ of f

But how does this help us?

If f and f’ are rewritten as:

$$f = \sum_{k \leq 8} \sum_{j \in Z^2(k)} \sum_{\psi \in \Psi} c_{j,k,\psi} \psi_{j,k}$$

$$f' = \sum_{k \leq 8} \sum_{j \in \mathbb{Z}^2(k)} \sum_{\psi \in \Psi} c'_{j,k,\psi} \psi_{j,k}$$

But how does this help us?

Accurate compression → Small differences in coefficients $c_{j,k}$ and $c'_{j,k}$

Efficient compression → Small number of non-zero coefficients

Rephrasing the question

New goal: Minimize the expression

$$\| f - f' \|_{L^p(\Omega)}$$

(with an f’ having **very few** non-zero coefficients in its Haar wavelet expansion)

Rephrasing the question

Specifically, take f’ whose non-zero coefficients agree with f

$$\tilde{f} = \sum_{(j,k,\psi)\in\Lambda} c_{j,k,\psi}\psi_{j,k} + P_0 f,$$

Where Λ is some finite set
(**Note:** size of Λ controls efficiency of compression)

Rephrasing the question

Taking **p = 2:**

$$\tilde{f} = \sum_{(j,k,\psi)\in\Lambda} c_{j,k,\psi}\psi_{j,k} + P_0 f,$$

Where Λ is some finite set
(**Note:** size of Λ controls efficiency of compression)

Rephrasing the question

$$\|f - f'\|^2_{L^2(\Omega)} = \sum_{(k,j,\psi)\notin\Lambda} |c_{j,k,\psi}|^2 \|\psi_{j,k}\|^2_{L^2(\Omega)}$$

$$= \sum_{(k,j,\psi)\notin\Lambda} |c_{j,k,\psi}|^2$$

Rephrasing the question

So if our goal is to ***minimize*** the sum

While keeping the size of Λ constant…

(Say $|\Lambda|$ = N)

$$\sum_{(k,j,\psi)\notin\Lambda} |c_{j,k,\psi}|^2$$

Rephrasing the question

So if our goal is to ***minimize*** the sum

$$\sum_{(k,j,\psi)\notin\Lambda} |c_{j,k,\psi}|^2$$

While keeping the size of Λ constant…

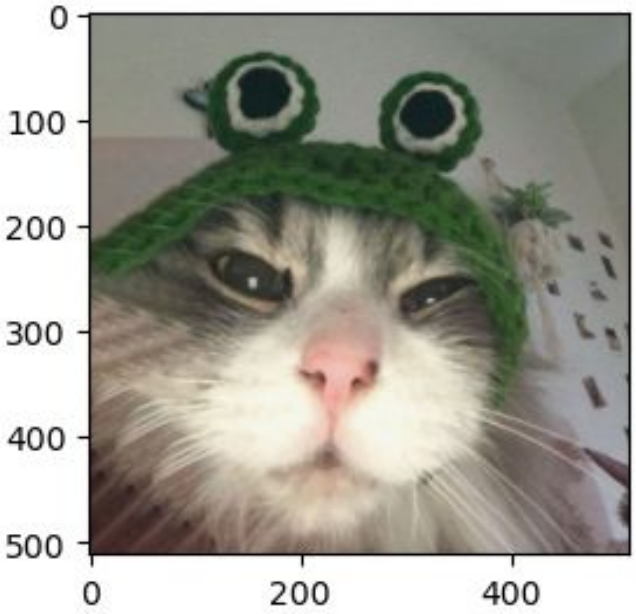
(Say $|\Lambda| = N$)



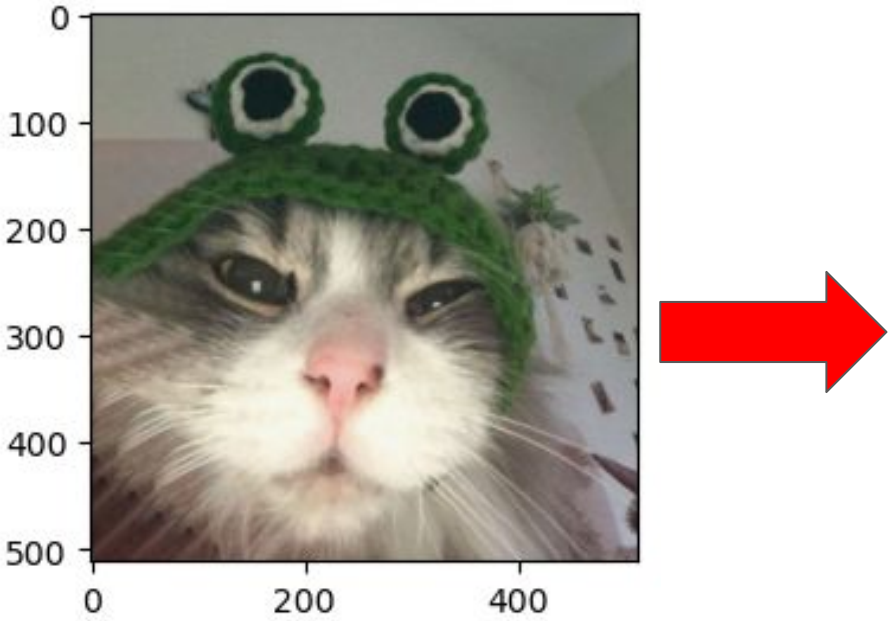
Answer: Take Λ to be the **N largest coefficients!**

Example

Example

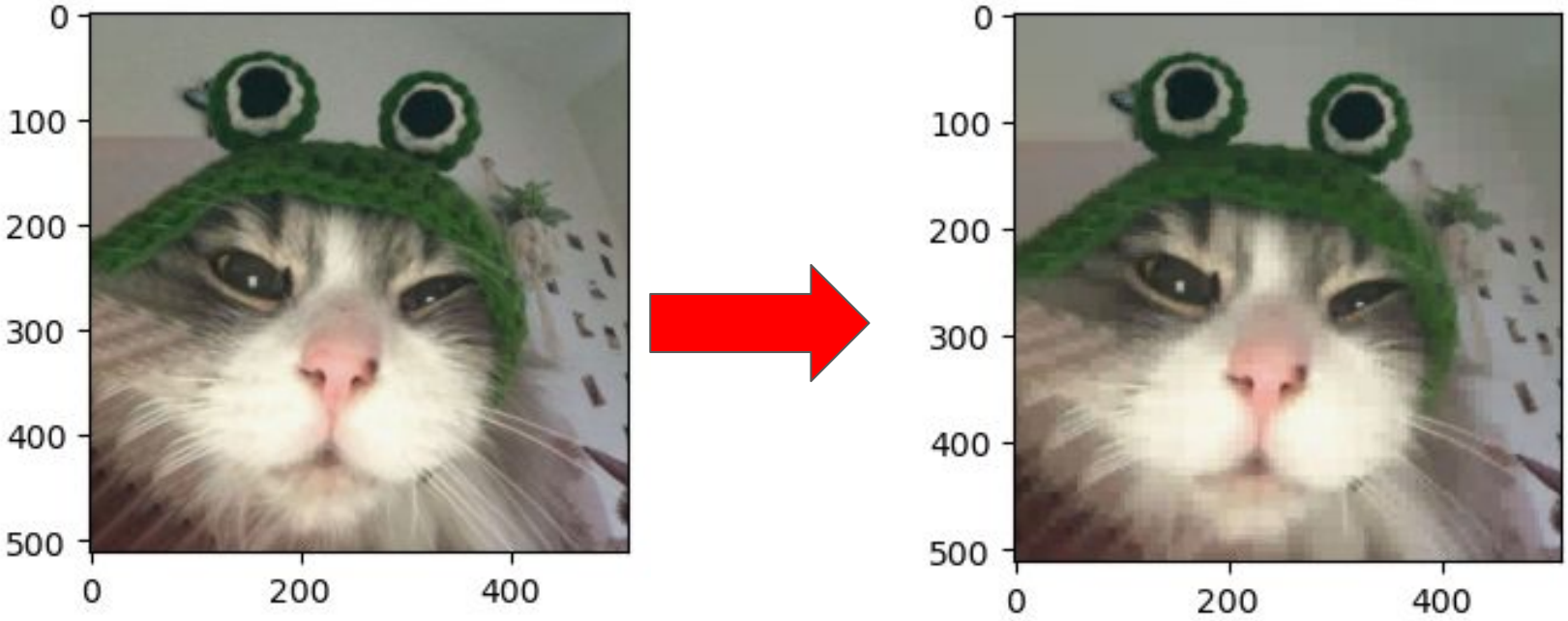


Example



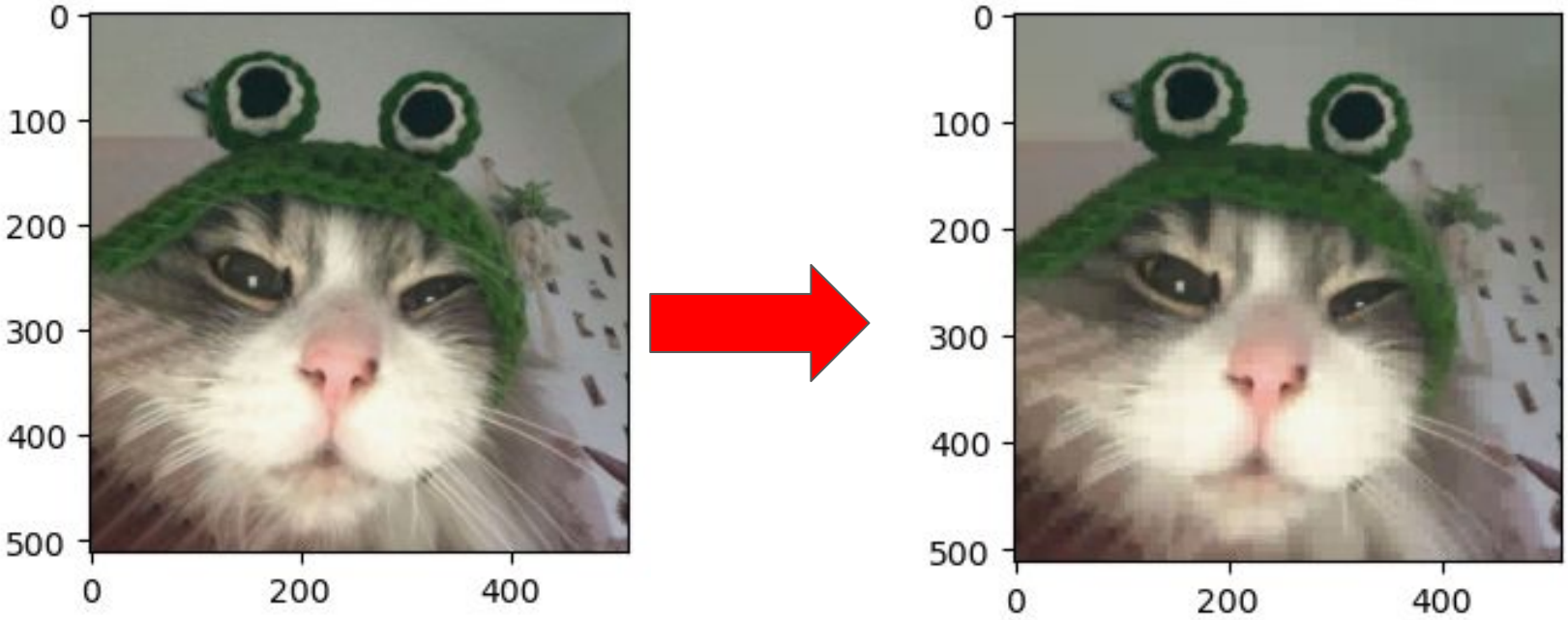
Keep only **N** largest values of Haar representation

Example



Keep only **N** largest values of Haar representation (can you guess what **N** is?)

Example



N = 4096 → (have achieved a 64:1 compression rate!)

Shortcomings of our method

- Finding the **N** largest values requires *computing* and *sorting* all the possible coefficients $\mathbf{c_{j,k,\psi}}$

Shortcomings of our method

- Finding the **N** largest values requires *computing* and *sorting* all the possible coefficients $\mathbf{c_{j,k,\psi}}$
  - (In practice - sorting this set takes *longer* than it does to compute all coefficients)

Alternate approach

- Start with threshold value $\varepsilon > 0$
- Keep only coefficients satisfying $c_{j,k,\psi} > \varepsilon$

Alternate approach

- Start with threshold value **$\varepsilon > 0$**
- Keep only coefficients satisfying **$c_{j,k,\psi} > \varepsilon$**

In fact, what is *even better* is to start with function defined by coefficients:

$$\tilde{c}_{j,k,\psi} = \text{round}\left(\frac{c_{j,k,\psi}}{2\epsilon}\right) 2\epsilon,$$

Where round(x) = nearest integer to x (these are called quantized coefficients)

Quick aside on quantized coefficients

- These satisfy a very nice property:

$$|c_{j,k,\psi} - \tilde{c}_{j,k,\psi}| \leq \epsilon$$

$$\text{if } |c_{j,k,\psi}| < \epsilon, \text{ then } \tilde{c}_{j,k,\psi} = 0.$$

Quick aside on quantized coefficients

- However, this is **not the main reason** why it is preferred to choose an approximating function with quantized coefficients

Quick aside on quantized coefficients

- However, this is **not the main reason** why it is preferred to choose an approximating function with quantized coefficients
- Given the *right choice of normed space*, one can actually use obtain a relation between N and ε

Quick aside on quantized coefficients

- However, this is **not the main reason** why it is preferred to choose an approximating function with quantized coefficients
- Given the *right choice of normed space*, one can actually use obtain a relation between N and ε
- The **Besov spaces** $B_q^n(L^p(\Omega))$ = "L^p functions with n 'derivatives' "
- If f belongs to such a space, then the **number of non-zero coefficients of ($c_{j,k,\psi}$')** (denoted N) satisfies

$$N \leq \epsilon^{-\tau} \|f\|^{\tau}_{B^{\alpha}_{\tau}(L^{\tau}(\Omega))}.$$

Generalizing to other wavelets

Generalizing to other wavelets

- Previously, we used 2-dim **Haar** wavelet - but there’s no reason to limit ourselves to this!

Generalizing to other wavelets

- Previously, we used 2-dim **Haar** wavelet - but there's no reason to limit ourselves to this!
- All we need is a family Ψ of mother wavelets, so that we can decompose arbitrary f into

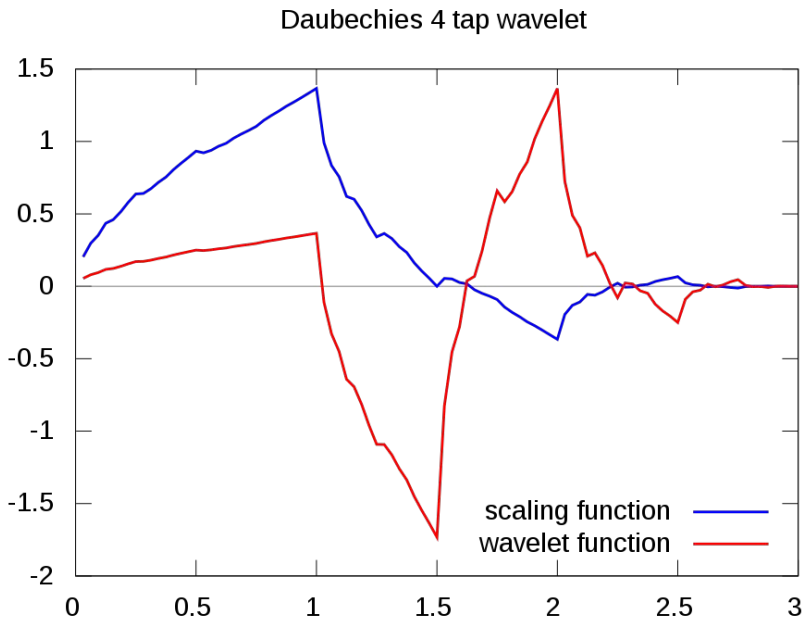
$$f = \sum_{k \geq 0} \sum_{j \in \mathbf{Z}^2(k)} \sum_{\psi \in \Psi} c_{j,k,\psi} \psi_{j,k}$$

Generalizing to other wavelets

- Some options:

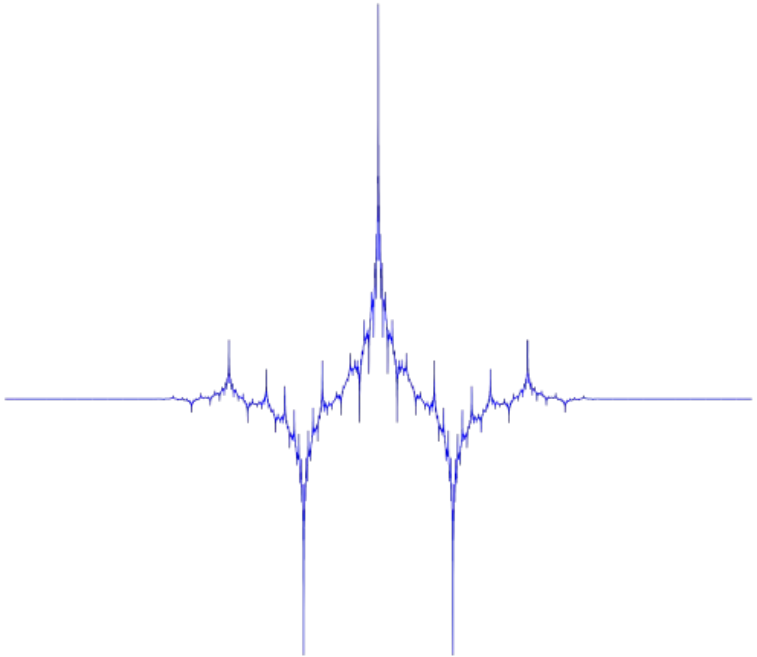
Generalizing to other wavelets

- Some options:
  - Daubechies-4 wavelet



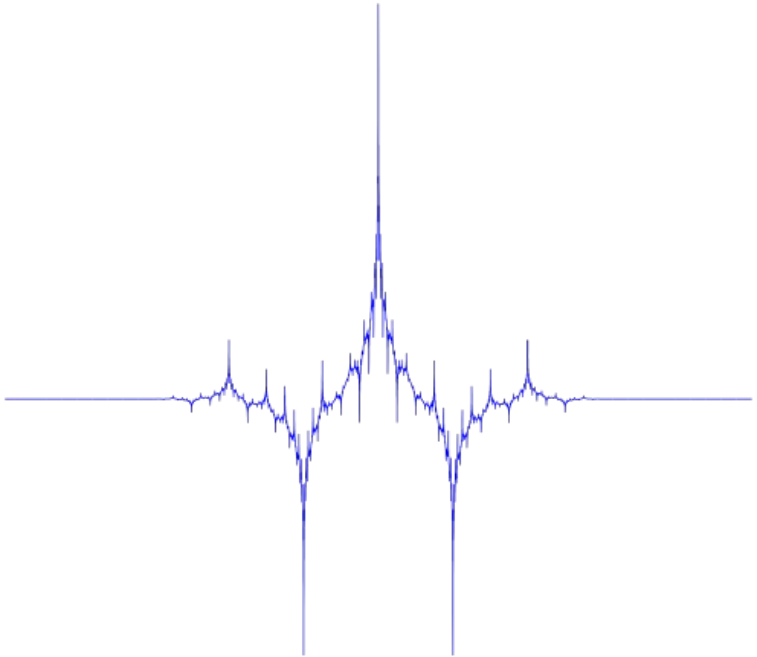
Generalizing to other wavelets

- Some options:
  - Daubechies-4 wavelet
  - Cohen-Daubechies-Feauveau 9/7
    - Used by JPEG2000



Generalizing to other wavelets

- Some options:
  - Daubechies-4 wavelet
  - Cohen-Daubechies-Feauveau 9/7
    - Used by JPEG2000
  - Cosine/Sine series
    - Used by JPEG, HEIC

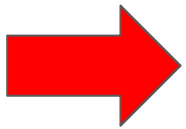


Understanding the Discrete Wavelet Transform (DWT)

Understanding the Discrete Wavelet Transform (DWT)

- One might expect a code implementation to look something like…

```
disc_wave_tf(image, 'haar')
```

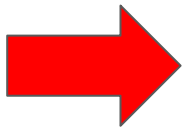


$[c_{0,0}, c_{0,1}, \ldots]$

Understanding the Discrete Wavelet Transform (DWT)

- One might expect a code implementation to look something like…

```
disc_wave_tf(image, 'haar')
```



$[c_{0,0}, c_{0,1}, \ldots]$

In reality, it is not so straightforward… but for good reason!

Understanding the Discrete Wavelet Transform (DWT)

- Two principles:
  - Don’t compute more coefficients than necessary (--> multilayer DWT)
  - Compute coefficients in an efficient, recursive manner

Understanding the Discrete Wavelet Transform (DWT)

- Two principles:
  - Don’t compute more coefficients than necessary (--> multilayer DWT)
  - Compute coefficients in an efficient, recursive manner

- In particular:
  - Avoid computing integrals!
  - Often find ourselves looking at data which is “not quite” the coefficients $\mathbf{c}_{j,k,\psi}$

Simplest Example of DWT

Consider a 1-dimensional array [7, 3, 4, 2, 5, 5, 1, 6] (equiv. f: [0,1] → R)

Simplest Example of DWT

Consider a 1-dimensional array [7, 3, 4, 2, 5, 5, 1, 6] (equiv. f: [0,1] → R)

$$\psi = \chi_{[0,1/2)} - \chi_{[1/2,1)}$$

(mother wavelet)

Simplest Example of DWT

Consider a 1-dimensional array [7, 3, 4, 2, 5, 5, 1, 6] (equiv. f: [0,1] → R)

$$\psi = \chi_{[0,1/2)} - \chi_{[1/2,1)}$$

(mother wavelet)

Smallest relevant wave packets:

$$\psi_{j,2} = 4(\chi_{[0,1/8)+j} - \chi_{[1/8,1/4)+j})$$

Simplest Example of DWT

Rather than compute the integral:

$$c_{0,2} = \int_{[0,1]} f(x)\psi_{0,2}(x)\,dx$$

Simplest Example of DWT

Rather than compute the integral:

$$c_{0,2} = \int_{[0,1]} f(x)\psi_{0,2}(x)\,dx$$

Instead, use the fact that f = [7, 3, ….]

$$\int f(x)\psi_{0,2}(x)\,dx = 4\left(\int_{[0,1/8)} 7\,dx - \int_{[1/8,1/4)} 3\,dx\right) = [\frac{1}{2}, -\frac{1}{2}] \cdot [7, 3]$$

Simplest Example of DWT

Second crucial detail to remember:

$$7 = 8\int_{[0,1/8)} f(x)\,dx = 4\int_{[0,1/8)} f(x)\,dx + 4\int_{[0,1/8)} f(x)\,dx \pm 4\int_{[1/8,1/4)} f(x)\,dx$$

$$= 4\int_{[0,1/4)} f(x)\,dx + \int_{[0,1/4)} \psi_{0,2}(x) f(x)\,dx$$

Simplest Example of DWT

Second crucial detail to remember:

$$7 = 8\int_{[0,1/8)} f(x)\,dx = 4\int_{[0,1/8)} f(x)\,dx + 4\int_{[0,1/8)} f(x)\,dx \pm 4\int_{[1/8,1/4)} f(x)\,dx$$

$$= 4\int_{[0,1/4)} f(x)\,dx + \int_{[0,1/4)} \psi_{0,2}(x) f(x)\,dx$$

$= a_1 + b_1$

Note: a_1 = [½, ½] * [7, 3]

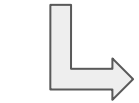
In other words - we can *reobtain* the first two values of f from these two coefficients

Simplest Example of DWT

[7, 3, 4, 2, 5, 5, 1, 6]

Simplest Example of DWT

[7, 3, 4, 2, 5, 5, 1, 6]



Compute:

$$a_1 = [\frac{1}{2}, \frac{1}{2}] \cdot [7, 3] = 5$$

$$b_1 = [\frac{1}{2}, -\frac{1}{2}] \cdot [7, 3] = 2$$

Simplest Example of DWT

[7, 3, 4, 2, 5, 5, 1, 6]



Repeat computation
with [4, 2]

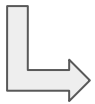
Keep track of coefficients:
[5, 0, 0, 0, 2, 0, 0, 0]

Simplest Example of DWT

[7, 3, 4, 2, 5, 5, 1, 6]



Repeat computation
with [4, 2]



$a_2 = 2, b_2 = 1$

Keep track of coefficients:
[5, 0, 0, 0, 2, 0, 0, 0]

Simplest Example of DWT

[7, 3, 4, 2, 5, 5, 1, 6]



$a_3 = 5, b_3 = 0$

Keep track of coefficients:
[5, 2, 0, 0, 2, 1, 0, 0]

Simplest Example of DWT

[7, 3, 4, 2, 5, 5, 1, 6]

$a_4 = 7/2, b_4 = -5/2$

Keep track of coefficients:
[5, 2, 5, 0, 2, 1, 0, 0]

Simplest Example of DWT

[7, 3, 4, 2, 5, 5, 1, 6]



[5, 2, 5, 7/2] , [2, 1, 0, -5/2]

Approximation Coefficients (Low Pass Filter)

Detail Coefficients (Low Pass Filter)

Simplest Example of DWT

[7, 3, 4, 2, 5, 5, 1, 6]



[5, 2, 5, 7/2] , [2, 1, 0, -5/2]

Approximation Coefficients (Low Pass Filter)

Detail Coefficients (Low Pass Filter)

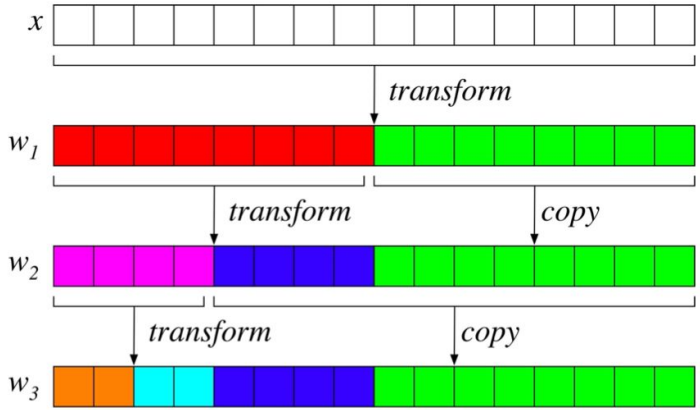
And as a nice fact - we can **invert** this transformation

DWT more generally

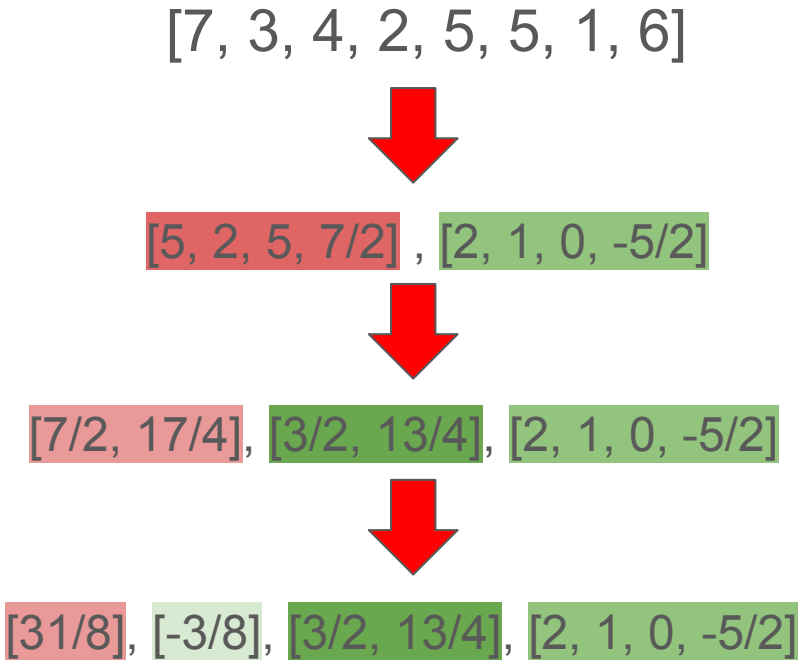
- We can apply our transform again, this time to the approximation coefficients!

DWT more generally

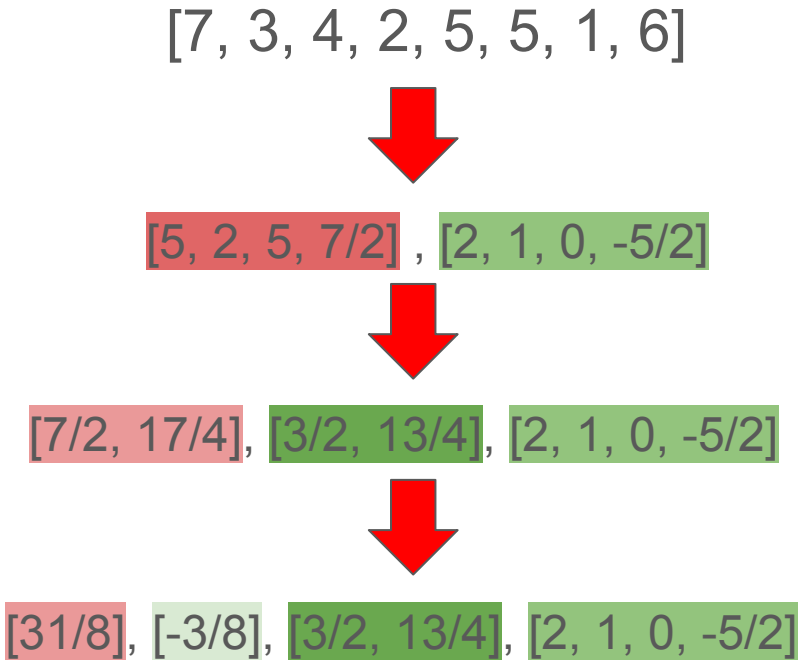
- We can apply our transform again, this time to the approximation coefficients!
- In general, can keep going until we reach the ‘minimal’ array size (depends on wavelet transform)



Our Example (cont.)

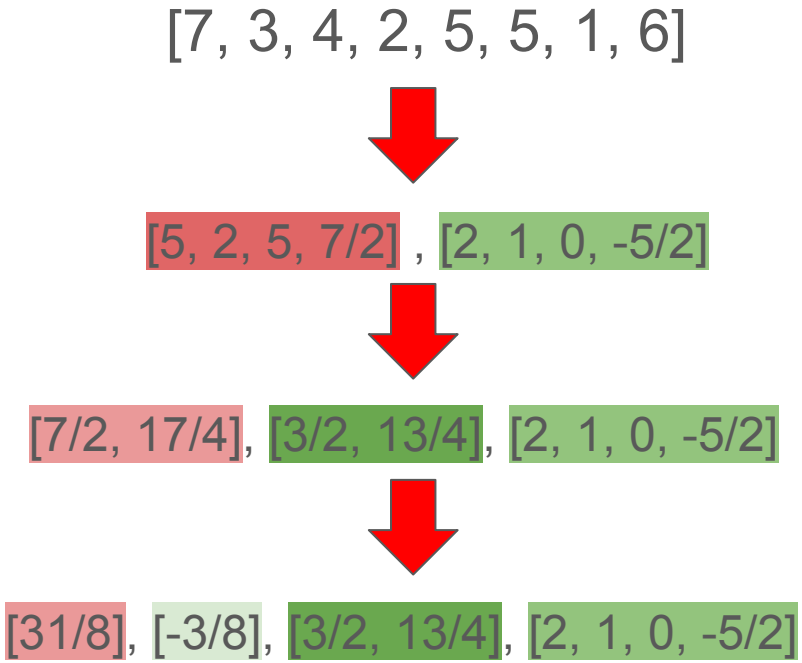


Our Example (cont.)



At this point, have to **stop**!

Our Example (cont.)



At this point, have to **stop**!

Note: these final numbers are our $\mathbf{c}_{j,k}$ - i.e. the coefficients of our wavelet transform of f

Final remarks on DWT

- One has the power to compute as many/as little coefficients as desired
- Even after multiple applications, is still invertible
- Works well as an implementation for the theoretical ideas discussed in the first half

References to check out

- List of papers at https://www.math.purdue.edu/~lucier/692/related_papers_summaries.html
  - Great resource for learning more about topics which are closer to research-side
- PyWavelet - https://pywavelets.readthedocs.io/en/latest/
  - Python module for implementing DWT / other wavelet transforms
  - (I can also send out a github link for a notebook I made detailing the examples included in this talk)
- https://parametric.press/issue-01/unraveling-the-jpeg/
  - Great article (~10 min read)